scripts/capture-workable-items-db-delta.sh â€” differential SQLite dump evidence for the workable-items SSoT

Revision: 1 **Last modified:** 2026-08-18T20:15:00Z **Status:** active **Item:** task #79 (Â§11.4.209 review IMPORTANT-2 remedy)

Overview

`scripts/capture-workable-items-db-delta.sh` produces a **differential SQLite dump** of `docs/workable_items.db` â€” the Â§11.4.93/Â§11.4.95 workable-items single-source-of-truth database â€” between a given commit and its first parent. The output is a human-readable, git- trackable evidence file under `docs/qa/db-deltas/<full-commit-sha>.diff` recording:

- per-table row counts, before and after, with a `same` / `CHANGED` marker per table;
- the full content of the `meta` table on both sides;
- a unified diff of the complete logical `.dump` of the database on both sides â€” the actual `INSERT`/`UPDATE`-equivalent row-level changes, not just â€œthe bytes differâ€.

Why this exists â€” the Â§11.4.209 review IMPORTANT-2 finding

Commit `3520621` closed BOB-108 and committed `docs/workable_items.db` alongside the doc regeneration it authorized. The commit message honestly stated *â€œmeta table content is unchangedâ€* â€” but that is a claim about **one table only**, and no differential evidence accompanied it. Per Â§11.4.226 (evidence-class-at-closure), a committed binary blob is an opaque **artifact-class** fact until something opens it and shows what changed â€” a message-level prose claim never rises above that class on its own.

The risk was not hypothetical: the **same sessionâ€™s** Task #41 investigation (`docs/incidents/2026-08-18-auto-committer-BOB-068-investigation.md`) independently confirmed that `commit-push-all.shâ€™s` default (unscoped) mode stages the **entire shared working tree at-instant**, including whatever a concurrent subagentâ€™s own `workable-items` invocation had put into `docs/workable_items.db` moments earlier â€” exactly the kind of race that would let an undisclosed mutation ride along inside a binary blob nobody inspected.

This script closes that gap going forward, and was used to backfill retroactive evidence for 3520621 itself (see docs/qa/db-deltas/3520621866e071a6d10ec06e5b432188fdac7129.diff).

Backfill result for 3520621 â€” the differential dump proves the binary blob change was NOT pure housekeeping as narrowly implied by the commit message: two rows in items actually changed (BOB-108 Queuedâ†’Fixed, the intended mutation; and BOB-104 Queuedâ†’In progress with an expanded body â€” an existing itemâ€™s status update that the commit message never mentioned), plus two new, correctly attributed audit rows in item_history (ids 89â€“90, By='AI', timestamped). Every other table (meta, logic_groups, test_diary, test_diary_summary, obsolete_details, operator_block_details, firebase_metadata) is byte-for-byte unchanged. **Verdict: no corruption, no data loss, every row-level change is legitimately audit-trailed in item_history and consistently reflected in docs/Issues.md/docs/Fixed.md** â€” but the commit messageâ€™s own scope description was incomplete (it silently absorbed the BOB-104 update without naming it), which is itself the class of finding Â§11.4.238 requires a discovery-ledger entry for. See docs/QA_DISCOVERY_LEDGER.md entry DB-BLOB-COMMITTED-WITHOUT-DELTA-3520621.

Usage

```
# Capture the delta for the current HEAD commit
scripts/capture-workable-items-db-delta.sh

# Capture the delta for a specific commit (any commit-ish git accepts)
scripts/capture-workable-items-db-delta.sh 3520621

# Force-regenerate a delta that already exists at that commit sha
scripts/capture-workable-items-db-delta.sh 3520621 --force
```

Output always lands at docs/qa/db-deltas/<full-commit-sha>.diff â€” named after the *resolved* full SHA, regardless of how the commit was specified on the command line, so the path is stable and collision- free.

Exit codes

Code	Meaning
0	Delta captured, OR honestly SKIPPED with a printed reason (Â§11.4.3)
1	Invocation error (bad arguments / unresolvable commit-ish)
2	-h/--help requested

Idempotence

If `docs/qa/db-deltas/<sha>.diff` already exists, the script prints a skip notice and exits 0 without touching the file â€” pass `--force` to regenerate. This makes it safe to call unconditionally from an automated wrapper (see below) without accumulating duplicate work or clobbering a prior evidence file on every re-run.

Honest skip conditions (Â§11.4.3)

- `sqlite3` not found on `PATH` â€” the script refuses to fabricate a delta and prints an honest skip reason instead. This is a **hard dependency**, not an optional enhancement: without it, no differential evidence can honestly be produced.
- `docs/workable_items.db` does not exist at the target commit â€” skip (nothing to diff).
- The target commit is the repository root (no parent) or its parent never tracked the DB â€” the delta is generated as a full dump of the AFTER side only, clearly labeled as having no BEFORE side.

Atomic evidence-file publication

The full capture (row counts + meta dump + unified `.dump` diff) is built into a temp file first and only `mv`-ed into place at the real output path once the ENTIRE capture succeeds. A mid-capture failure (e.g.Â a broken `sqlite3` invocation) therefore **never** leaves a partial, truncated file sitting at the real evidence path â€” that would read as â€œclean, nothing after this lineâ€ instead of the genuine failure it is, precisely the false-negative-null class Â§11.4.201(6) forbids. This was verified with a real Â§1.1 paired mutation: breaking the `sqlite3 ... .dump` call inside a scratch copy of the script produces exit 127 and **zero files** at the real output path; the unmodified script then reproduces the full evidence file cleanly (Â§11.4.115 REDâ†’GREEN polarity confirmed in the same session, task #79).

Wiring â€” `scripts/commit-push-all.sh` stage 5.5

`scripts/commit-push-all.sh` invokes this helper automatically as its **stage 5.5**, immediately after a commit lands in stage 5 and before the push stage (stage 6). It only fires when:

1. Stage 5 actually created a NEW commit (HEAD moved), AND
2. That new commitâ€™s diff against its parent touches `docs/workable_items.db`.

When it fires, the helper is invoked for the just-created `HEAD`. If it produces a new (not-already-tracked-identically) delta file, that file is staged and landed as an **immediate, narrowly-scoped follow-up commit** â€” `docs(qa,db-delta): capture differential dump for HEAD <short-sha>` â€” so both

commits travel to every configured upstream together in the same `commit-push-all.sh` invocation (stage 6 still pushes whatever `HEAD` is at that point).

Capture failure at this stage (e.g.Â `sqlite3` genuinely absent on the host) is a **non-blocking WARN**, never a hard failure of the commit/push mechanism â€” per Â§11.4.234(D)â€™s always-unblocked invariant, an evidence-capture helper must never be able to make the dedicated commit/push entrypoint unusable. The gap is printed loudly to stderr so it is never silently lost.

Related documents

- `scripts/commit-push-all.sh` (the wrapper this helper is wired into)
- `docs/scripts/commit-push-all.md` (the wrapperâ€™s own doc)
- `docs/QA_DISCOVERY_LEDGER.md` â€” entry `DB-BLOB-COMMITTED-WITHOUT-DELTA-3520621`
- `.superpowers/sdd/task-review-457cca4-a7e55f9-report.md` â€” the Â§11.4.209 review finding (IMPORTANT-2) this script remedies
- `docs/incidents/2026-08-18-auto-committer-BOB-068-investigation.md` â€” Task #41â€™s independent confirmation of the shared-checkout race that makes this evidence class load-bearing